Learning Objectives

By the end of this session, you should be able to:

- Extend our compiler's grammar from individual expressions to complete programs containing function definitions.
- Reuse the expression parser while implementing a parser for function definitions, compound statements, and expression statements.
- Understand how recursive-descent parsing can generate assembly code directly, without constructing a syntax tree for every language construct.
- Recognize how keywords and punctuators are handled by the lexer and used by the parser.
- Generate executable ULM programs containing multiple functions that can call one another.
- Understand how different parts of a grammar can be handled by separate parser modules, using either expression trees or direct code generation.
- Extend the expression parser to support character literals using tokens already recognized by our lexer.

Keeping Your Work

In this worksheet, we will continue working on our compiler project. All files should be created or modified directly in your `not-abc` repository.

Step 1 — Our First *not-abc* Program

Let's start with the simplest possible not-abc program. Create a subdirectory `examples` and, inside it, a file named `ws18_ex1.not-abc` containing:

```
fn main()
{
}
```

This will be our first test program. It does nothing, but by the end of this worksheet, our compiler should be able to translate it into executable ULM assembly code.

Next, create a second file, `examples/ws18_ex2.not-abc`, containing:

```
fn main()
{
    putchar(65);
    putchar(10);
}
```

This will be our next test case. Once our compiler supports expression statements and function calls, compiling and executing this program should print the letter `A`, followed by a newline.

1. **Check your editor.** If you use Neovim with the configuration provided for this course, you should already get syntax highlighting for `.not-abc` files. If you use another editor and do not get syntax highlighting, please let us know.

2. **Check your lexer.** Run `make` to rebuild all the compiler components, then use the lexer test program:

   ```
   make
   ./xtest_lexer < examples/ws18_ex1.not-abc
   ```

   The output should begin with:

   ```
   1.1: FN (fn)
   1.4: IDENTIFIER (main)
   1.8: LPAREN (()
   1.9: RPAREN ())
   ...
   ```

   Inspect the complete output. There should be no `BAD_TOKEN`, and the keyword `fn` must be recognized as `FN`, not as an ordinary identifier.

   If this does not work, revisit Worksheet 15 before proceeding.

   Notice that our lexer already recognizes everything needed for this program: the keyword `fn`, identifiers, parentheses, braces, semicolons, and integer literals. We implemented these features in earlier worksheets. Now it's finally time to put them to work!

3. **Don't forget Git!** Before moving on, add your first not-abc program to the repository:

   ```
   git add examples/ws18_ex*.not-abc
   ```

   We'll return to this program once our new parser is ready.

© Michael C. Lehn | CC BY-SA 4.0

Step 2 — Setting Up Our Parser

Our lexer already recognizes all the tokens needed for our first *not-abc* program. Now we build a parser that implements the following grammar:

```
top-level     = { fn } EOI
fn            = "fn" identifier "(" ")" compound-stmt
compound-stmt = "{" { expr-stmt } "}"
expr-stmt     = expr ";"
```

Here, `expr` is the expression grammar implemented in Worksheet 17. We will reuse that parser rather than implement it again.

Tasks

1. **Create the compiler's main program.** Create `not-abc.abc` with the following content:

   ```
   @ <stdlib.hdr>
   @ <stdio.hdr>
   @ "gen.hdr"
   @ "parser.hdr"

   fn main()
   {
       genInit();
       if (!parseTopLevel()) {
           fprintf(stderr, "expected end of input\n");
           exit(1);
       }
   }
   ```

   Unlike our `xtest_*.abc programs`, this will become the actual executable for our *not-abc* compiler.

   The function `parseTopLevel()` will return `true` if the input is a valid *not-abc* program and `false` otherwise. Notice that we use `fprintf(stderr, ...)` to report an error. Unlike `printf`, which writes to standard output (`stdout`), this sends the message to standard error (`stderr`). We will see shortly why this distinction matters for a compiler.

2. **Define the parser interface.** Create `parser.hdr` containing:

   ```
   extern fn parseTopLevel(): bool;
   ```

   This function will be the entry point for parsing an entire *not-abc* program.

3. **Create the parser implementation.** Create `parser.abc` with the following stub:

   ```
   @ "lexer.hdr"
   @ "parser.hdr"

   fn parseTopLevel(): bool
   {
       getToken();
       return false;
   }
   ```

   For now, `parseTopLevel()` is just a stub that always returns false. In the following steps, we will gradually develop both the grammar and its implementation, starting with the simplest possible rule and working our way towards the complete grammar shown above.

4. **Build the compiler.** Run `make` and verify that the new not-abc executable is built successfully.

5. **Run the compiler.** Let's see what our compiler does so far. Run:

   ```
   ./not-abc < examples/ws18_ex1.not-abc
   ```

   You should see some generated ULM assembly code, followed by the error message:

   ```
   expected end of input
   ```

   Now redirect the output to a file:

   ```
   ./not-abc < examples/ws18_ex1.not-abc > examples/ws18_ex1.s
   ```

   This time, only the error message appears in the terminal. The generated assembly code is written to `examples/ws18_ex1.s`. Our compiler uses two separate output streams:

   - **Standard output (`stdout`)** is used for generated assembly code. The operator `>` redirects it to a file.
   - **Standard error (`stderr`)** is used for error messages. It is not affected by this redirection and still appears in the terminal.

   This separation is particularly useful for a compiler: we can save its generated code without mixing it with error messages.

© Michael C. Lehn | CC BY-SA 4.0

Step 3 — Our First Grammar Rule

Let's start with the simplest possible grammar:

```
top-level = EOI
```

The only valid not-abc program is therefore an empty file. However, our compiler does not even recognize this trivial grammar yet.

Tasks

1. **Test the current compiler.** Run:

   ```
   echo "" | ./not-abc
   ```

   Our compiler still reports expected end of input, even though the input contains nothing but a newline.

   While we're at it, let's learn another useful Unix trick:

   ```
   echo "" | ./not-abc > /dev/null
   ```

   The special file `/dev/null` discards everything written to it. This allows us to suppress the generated assembly code while still seeing error messages on `stderr`.

   Now check the exit status

   ```
   echo $?
   ```

   It should be `1`, indicating that our compiler terminated with an error.

2. **Implement the grammar.** In `parser.abc`, change `parseTopLevel()` to:

   ```
   fn parseTopLevel(): bool
   {
       getToken();
       return token.kind == EOI;
   }
   ```

   Rebuild the compiler with make and repeat the tests from Task 1. This time, there should be no error message, and the exit status should be `0`.

   Also try compiling `examples/ws18_ex1.not-abc`. This program must still be rejected because our grammar does not yet allow function definitions.

   Although our compiler now accepts its first valid input, the generated assembly code is not particularly useful yet. The initialization code calls a function named `main`, but our empty program does not define it. If you try to assemble the generated code with `ulmas`, it will report an undefined symbol.

© Michael C. Lehn | CC BY-SA 4.0

Step 4 — Parsing Function Definitions

Let's extend our grammar to support function definitions with empty bodies:

```
top-level     = { fn } EOI
fn            = "fn" identifier "(" ")" compound-stmt
compound-stmt = "{" "}"
```

Our compiler should now accept programs such as examples/ws18_ex1.not-abc. For now, we only want to recognize valid programs. We will generate code for function definitions in the next step.

Tasks

1. **Prepare the parser.** Replace the contents of `parser.abc` with:

   ```
   @ "lexer.hdr"
   @ "parser.hdr"
   @ "parseexpr.hdr"

   fn parseFn(): bool;
   fn parseCompoundStmt(): bool;

   fn parseTopLevel(): bool
   {
       getToken();
       // TODO
       return token.kind == EOI;
   }

   fn parseFn(): bool
   {
       // TODO
       return false;
   }

   fn parseCompoundStmt(): bool
   {
       if (token.kind != LBRACE) {
           return false;
       }
       getToken();
       expected(RBRACE);
       getToken();
       return true;
   }
   ```

   Notice that `parseCompoundStmt()` is already implemented. It recognizes an empty compound statement consisting of `{` and `}`. We also reuse `expected()` from `parseexpr.abc`, introduced in Worksheet 17.

2. **Implement the grammar.** Implement `parseFn()` according to the grammar above, using the recursive-descent parsing techniques from Worksheet 13. Then extend `parseTopLevel()` to recognize zero or more function definitions followed by `EOI`.
   Remember that a parsing function returning bool should return `false` if the corresponding grammar rule does not begin at the current token. Once the rule has been recognized, required tokens can be checked using `expected()`.
   If you get stuck, feel free to ask for help. And don't worry too much: in the next step, we will extend the parser to generate code, and the instructions will indirectly reveal a possible solution.

3. **Test your parser.** Rebuild the compiler and try:

   ```
   ./not-abc < examples/ws18_ex1.not-abc > examples/ws18_ex1.s
   ```

   The compiler should now accept this program without reporting an error. Also check that empty input is still accepted.
   Try modifying the program by removing a parenthesis or a brace. Does your parser detect the error?
   The generated assembly code still does not contain a definition of `main`. That's expected: so far, we are only parsing function definitions, not generating code for them.

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — From Parsing to Code Generation

Let's compare your implementation with a possible solution shown below. If your parser already works, there is no need to replace it. Just check whether you missed anything.

```
@ <stdio.hdr>
@ <stdlib.hdr>
@ "lexer.hdr"
@ "parser.hdr"
@ "parseexpr.hdr"

fn parseFn(): bool;
fn parseCompoundStmt(): bool;

fn parseTopLevel(): bool
{
    getToken();
    while (parseFn()) {
    }
    return token.kind == EOI;
}

fn parseFn(): bool
{
    if (token.kind != FN) {
        return false;
    }
    getToken();
    expected(IDENTIFIER);
    // TODO
    getToken();
    expected(LPAREN);
    getToken();
    expected(RPAREN);
    getToken();
    if (!parseCompoundStmt()) {
        fprintf(stderr, "expected compound statement\n");
        exit(1);
    }
    // TODO
    return true;
}

fn parseCompoundStmt(): bool
{
    if (token.kind != LBRACE) {
        return false;
    }
    getToken();
    expected(RBRACE);
    getToken();
    return true;
}
```

Notice how closely the implementation follows our grammar. The repetition `{ fn }` becomes a `while` loop, and each grammar rule has its own parsing function.
Also notice the additional headers `stdio.hdr` and `stdlib.hdr`, needed for `fprintf()` and `exit()`. Whenever you use a new function, you should get into the habit of checking which header declares it. Don't wait for the compiler to remind you!
Our parser can now recognize function definitions, but it still does not generate code for them. The two TODO comments already suggest where code generation should take place.

Tasks

Take a look at `genFuncDefBegin()` and `genFuncDefEnd()` from Worksheet 14. Use them to extend your parser so that it also generates code for function definitions. The TODO comments indicate where these functions might be called. Rebuild the compiler and test it with:

```
./not-abc < examples/ws18_ex1.not-abc > examples/ws18_ex1.s
ulmas examples/ws18_ex1.s
ulm a.out
```

Your first not-abc program should now execute successfully, although it does nothing visible yet. A possible solution will be shown in the next step.

© Michael C. Lehn | CC BY-SA 4.0

Step 6 — Putting It All Together

Let's first look at a possible implementation of `parseFn()` that also generates code:

```
fn parseFn(): bool
{
    if (token.kind != FN) {
        return false;
    }
    getToken();
    expected(IDENTIFIER);
    genFuncDefBegin(token.val);
    getToken();
    expected(LPAREN);
    getToken();
    expected(RPAREN);
    getToken();
    if (!parseCompoundStmt()) {
        fprintf(stderr, "expected compound statement\n");
        exit(1);
    }
    genFuncDefEnd();
    return true;
}
```

Notice that our parser now does more than recognize tokens: it generates assembly code while parsing. Unlike our expression parser, it does not construct a syntax tree for function definitions.
To implement the complete grammar introduced in Step 2, we still need to handle expression statements. Add the following function to `parser.abc`:

```
fn parseExprStmt(): bool
{
    local expr: -> Expr = parseExpr();
    if (!expr) {
        return false;
    }
    expected(SEMICOLON);
    getToken();
    local retVal: GenReg = loadExpr(expr);
    genReleaseReg(retVal);
    releaseExpr(expr);
    return true;
}
```

Here we reuse the expression parser from Worksheet 17. It constructs an expression tree, which `loadExpr()` translates into assembly code. Afterwards, we release both the result register and the expression tree. Finally, modify `parseCompoundStmt()`:

```
fn parseCompoundStmt(): bool
{
    if (token.kind != LBRACE) {
        return false;
    }
    getToken();
    while (parseExprStmt())
    {
    }
    expected(RBRACE);
    getToken();
    return true;
}
```

Tasks

1. **Organize your parser.** Arrange the parsing functions in `parser.abc` in the same order as the grammar rules above. Update the forward declarations.
2. **Understand the implementation.** Verify that each parsing function implements its corresponding grammar rule. Pay particular attention to how `parseExprStmt()` connects expression parsing with code generation.
3. **Compile and run a complete program.** Rebuild the compiler and try:
   ```
   ./not-abc < examples/ws18_ex2.not-abc > examples/ws18_ex2.s
   ulmas -o examples/ws18_ex2 examples/ws18_ex2.s
   ulm examples/ws18_ex2
   ```
   Your program should print `A`, followed by a newline.

© Michael C. Lehn | CC BY-SA 4.0

Step 7 — Character Literals: Almost for Free

Let's try something a little more ambitious. Create `examples/ws18_ex3.not-abc` with the following program:

```
fn bar()
{
    putchar('b');
    putchar('a');
    putchar('r');
    putchar('\n');
}

fn foo()
{
    putchar('f');
    putchar('o');
    putchar('o');
    bar();
    putchar('\n');
}

fn main()
{
    foo();
}
```

Tasks

1. **Find out where our compiler fails.** Try compiling the program:

   ```
   ./not-abc < examples/ws18_ex3.not-abc > examples/ws18_ex3.s
   ```

   You should get an error message similar to:

   ```
   3.13: 'RPAREN' expected
   ```

   Not exactly the most user-friendly error message! But it gives us a clue: when parsing `putchar('b')`, our parser encounters the token `CHARACTER_LITERAL` where it expects `RPAREN` (the closing parenthesis `)`).

   Our lexer already recognizes character literals, but our expression parser does not yet know what to do with them.

2. **Extend the expression grammar.** We want character literals to behave just like decimal literals. Extend the grammar rule for factor to:

   ```
   factor = decimal-literal
          | character-literal
          | identifier “(“ [ expr { “,” expr } ] ”)”
          | "(" expr ")"
   ```

   For our compiler, the value of a character literal is simply the numerical code of that character.

   And, almost miraculously, everything else is already prepared! In parseexpr.abc, find the following line in parseFactor():

   ```
   if (token.kind == DECIMAL_LITERAL) {
   ```

   Replace it with:

   ```
   if (token.kind == DECIMAL_LITERAL || token.kind == CHARACTER_LITERAL) {
   ```

   That's it! No changes to the lexer, expression trees, or code generator are needed.

3. **Compile and run the program.** Rebuild the compiler and try again:

   ```
   ./not-abc < examples/ws18_ex3.not-abc > examples/ws18_ex3.s
   ulmas -o examples/ws18_ex3 examples/ws18_ex3.s
   ulm examples/ws18_ex3
   ```

   The output should be: `foobar` followed by an additional newline.
   Notice how our two parser modules work together: `parser.abc` handles function definitions and statements, while `parseexpr.abc` handles expressions, including character literals. Each parser can be extended independently.

4. **Don't forget Git!** Add your new files, commit your changes, and push everything to GitHub:

   ```
   git add *.abc *.hdr examples/ws18_ex*.not-abc
   git commit -a -m "Worksheet 18 is done!"
   git push
   ```

© Michael C. Lehn | CC BY-SA 4.0